SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	S Aswini	**Reg. No.:**	192511077
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.

- Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.

3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description
   - Scope of the System
   - Functional Requirements
   - Non-Functional Requirements
   - Use Case Descriptions
   - Data Requirements
   - External Interface Requirements
   - System Features
5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement	Clearly identifies comprehensive, accurate, and well-classified functional	Identifies most requirements with minor	Identifies only basic requirements;	Requirements are incomplete, inaccurate, or

Identification (25%)	and non-functional requirements.	classification errors.	several omissions.	poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

1. Study and Analyze the Scenario

1.1 Introduction

The Digital Platform to Crowdsource Societal Challenges is a web-based system that allows citizens to report local and community problems, share ideas, discuss possible solutions, and collaborate with

organizations that can help address them. The platform brings public concerns together in one place so that challenges can be understood, prioritized, and tracked transparently.

1.2 Business Domain

The system belongs to the civic technology, e-governance, community engagement, and social innovation domain. It supports communication between citizens, community moderators, administrators, and partner organizations such as NGOs or local service groups.

1.3 Problem Description

Societal problems such as waste management, damaged roads, water shortages, public safety concerns, and lack of community facilities are often reported through disconnected channels. Information may be incomplete, repeated, difficult to track, or not visible to the people who can act on it. Citizens may not know whether their reports have been reviewed or what progress has been made. A centralized platform is needed to collect reports, encourage community participation, support prioritization, and show the status of each challenge.

1.4 Objectives

- Provide a common digital space for reporting and discussing societal challenges.
- Enable citizens to support, comment on, and follow challenges.
- Help moderators verify reports and reduce spam or inappropriate content.
- Help administrators and partner organizations prioritize challenges and publish progress updates.
- Improve transparency by displaying clear challenge statuses and outcome information.

1.5 Stakeholders and Their Needs

Stakeholder	Role / Need
Citizens / Community members	Submit challenges, upload supporting details, vote, comment, and track progress.
Moderators	Review submissions, flag duplicate or unsuitable content, and maintain community standards.
System administrator	Manage users, categories, access permissions, platform settings, and reports.
NGOs / Local authorities / Partners	Review relevant challenges, coordinate responses, and post action updates.
Development and maintenance team	Maintain security, reliability, usability, and future improvements.

1.6 Existing Challenges

- Reports are scattered across social media, phone calls, forms, and messaging groups.
- Duplicate reports make it difficult to understand the actual level of need.
- Citizens receive limited feedback about whether an issue is being handled.
- It is difficult to compare urgency and community support consistently.
- Unmoderated posts can contain spam, misleading information, or personal data.

2. Functional and Non-Functional Requirements

2.1 Functional Requirements

ID	Requirement	Description
FR-01	User registration and login	The system shall allow users to create accounts and sign in securely.
FR-02	Profile management	The system shall allow users to view and update permitted profile information.
FR-03	Submit a challenge	The system shall allow users to submit a title, description, category, location/area, and optional supporting image.
FR-04	Browse and search	The system shall allow users to browse and search challenges by keyword, category, location, status, and date.
FR-05	Vote and comment	The system shall allow authenticated users to support a challenge and add relevant comments.
FR-06	Follow a challenge	The system shall allow users to follow a challenge and view its status updates.
FR-07	Moderation	The system shall allow moderators to approve, reject, flag, or request clarification for submitted content.
FR-08	Duplicate handling	The system should help moderators identify similar reports and link duplicates to a main challenge.
FR-09	Status tracking	The system shall display statuses such as Submitted, Under Review, Accepted, In Progress, Resolved, and Closed.
FR-10	Partner updates	Authorized partner organizations shall be able to post progress notes and resolution evidence.
FR-11	Notifications	The system shall notify users about moderation decisions, comments, and status changes.
FR-12	Administration	Administrators shall manage users, categories, roles, reports, and system settings.
FR-13	Dashboard and reports	The system shall provide summaries of challenge counts, categories, statuses, participation, and resolution progress.
FR-14	Report abuse	Users shall be able to report harmful, misleading, or inappropriate content.

2.2 Non-Functional Requirements

Quality Attribute	Requirement

Performance	Common pages should load within 3 seconds under normal expected usage.
Security	Passwords must be securely hashed; role-based access control and protected sessions must be used.
Privacy	Only necessary personal data should be collected; private contact details should not be displayed publicly by default.
Usability	The interface should be clear, mobile-friendly, accessible, and understandable to first-time users.
Reliability	The system should preserve submitted reports and avoid losing data during normal operation.
Availability	The platform should target 99% monthly availability, excluding planned maintenance.
Scalability	The design should support growth in users, reports, comments, and uploaded files.
Maintainability	The application should use modular components, documented APIs, and version control.
Compatibility	The system should work on current versions of common desktop and mobile browsers.
Accessibility	Forms, buttons, and status indicators should be usable with keyboard navigation and readable labels.

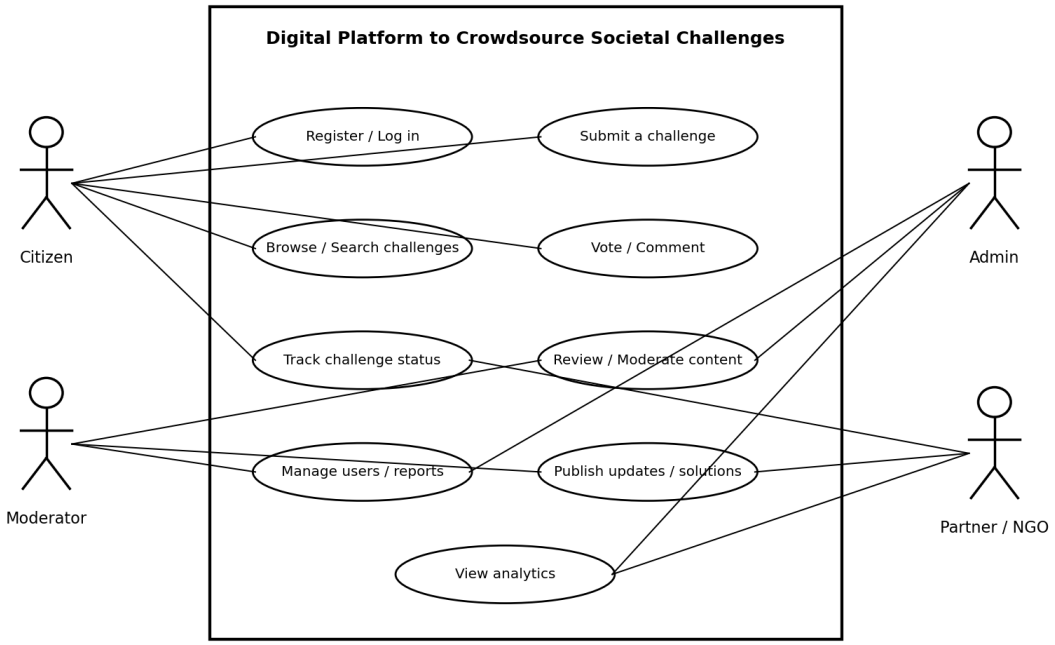
3. Actors and Use Cases

3.1 Actors

Actor	Type	Responsibilities
Citizen / Registered User	Primary	Register, submit challenges, search, vote, comment, follow, and report content.
Moderator	Primary	Review and moderate challenges, manage reports, and identify duplicates.
Administrator	Primary	Manage accounts, roles, categories, settings, and analytics.
Partner Organization	Primary	View assigned/relevant challenges and publish progress or solution updates.
Notification Service	Secondary external service	Deliver email or in-app notifications where configured.

3.2 Use Case Diagram

The diagram below summarizes the main interactions between platform actors and system functions.



3.3 Major Use Cases

Use Case	Actor(s)	Outcome
Register / Log in	Citizen, Moderator, Admin, Partner	The user obtains access according to their role.
Submit a challenge	Citizen	A new challenge is recorded with a Submitted status.
Browse / Search challenges	All permitted users	Relevant challenge details are displayed.
Vote / Comment / Follow	Registered Citizen	Community support and discussion are recorded.
Review / Moderate content	Moderator	Content is approved, rejected, flagged, or returned for clarification.
Update challenge status	Moderator, Admin, Partner (as authorized)	Progress is reflected on the challenge page.
Manage users and categories	Administrator	Platform access and classification are maintained.
View analytics	Administrator, authorized partners	Summary reports support planning and prioritization.

4. Software Requirement Specification (SRS)

4.1 Purpose

This SRS defines the expected features, interfaces, quality requirements, and constraints of the proposed crowdsourcing platform. It is intended for students, developers, testers, project supervisors, and stakeholders.

4.2 Scope of the System

The system will support challenge submission, discovery, community participation, moderation, status tracking, partner updates, notifications, and basic analytics. It will not itself guarantee that a government body or organization will resolve a reported issue; resolution depends on the responsible stakeholder.

4.3 Use Case Descriptions

Use Case 1: Submit a Societal Challenge

Field	Description
Primary actor	Citizen / Registered User
Precondition	The user is signed in.
Main flow	1. User selects Submit Challenge. 2. Enters title, description, category, area/location, and optional evidence. 3. System validates required fields. 4. User submits. 5. System saves the challenge and assigns Submitted status. 6. System confirms submission.
Alternative flow	If required details are missing or invalid, the system displays an error and asks the user to correct the form.
Postcondition	The challenge is stored and appears in the user's submissions; moderation may be required before public display.

Use Case 2: Moderate a Challenge

Field	Description
Primary actor	Moderator
Precondition	Moderator is authenticated and has moderation permission.
Main flow	1. Moderator opens the review queue. 2. Checks content and supporting details. 3. Chooses approve, reject, flag, or request clarification. 4. System records the decision. 5. User receives a notification where enabled.
Alternative flow	If the content may be a duplicate, the moderator links it to an existing challenge or requests more information.
Postcondition	The review decision and its timestamp are recorded.

Use Case 3: Publish a Progress Update

Field	Description
Primary actor	Authorized Partner / Administrator
Precondition	The actor has permission to update the selected challenge.
Main flow	1. Actor opens a challenge. 2. Adds a progress note and optional evidence. 3. Selects an allowed status. 4. System validates and stores the update. 5. Followers are notified where enabled.
Alternative flow	If the actor lacks permission or the status transition is invalid, the system rejects the update and explains why.
Postcondition	The challenge timeline reflects the update.

4.4 Data Requirements

Data Entity	Key Data Fields
User	User_ID, name/display name, email, password hash, role, account status, created date
Challenge	Challenge_ID, title, description, category_ID, area/location, submitted_by, created date, status, priority
Comment	Comment_ID, challenge_ID, user_ID, text, created date, moderation status
Vote / Support	Vote_ID, challenge_ID, user_ID, created date; one active vote per user per challenge
Status Update	Update_ID, challenge_ID, updated_by, old status, new status, note, timestamp
Category	Category_ID, category name, description, active status
Content Report	Report_ID, reported_item_ID, reporter_ID, reason, status, reviewed_by
Attachment	Attachment_ID, challenge_ID, file reference, file type, upload date

4.5 External Interface Requirements

- User interface: responsive web pages for registration, login, challenge forms, search, dashboards, and status timelines.
- Software interface: application server communicates with a database through controlled data-access components or APIs.
- Notification interface: optional email or in-app notification service for account and challenge updates.
- File interface: controlled upload of supported image formats with size limits and file validation.
- Accessibility interface: clear labels, validation messages, readable contrast, and keyboard-operable controls.

4.6 Main System Features

- Community challenge submission and discovery.
- Discussion, voting, and following to show community interest.
- Moderation queue and abuse reporting.
- Challenge status timeline and partner progress updates.
- Administrative dashboard and summary analytics.
- Role-based access and privacy-aware user profiles.

5. Assumptions and Constraints

5.1 Assumptions

- Users have access to a smartphone or computer and an internet connection.
- Users provide truthful and relevant information when submitting challenges.
- Moderators and partner organizations review assigned items within a reasonable time.

- A challenge can be categorized and associated with a general area without exposing sensitive personal information.
- Email or in-app notifications are available if the notification feature is enabled.

5.2 Constraints

Constraint Type	Constraint / Impact
Technical	The system requires hosting, a database, secure authentication, backups, and controlled file storage.
Operational	Moderators and partners must be available to review reports and provide updates.
Economic	Hosting, storage, maintenance, and notification services may have costs; the project budget may limit scale.
Legal / Privacy	Personal data, images, location information, and user-generated content must be handled according to applicable rules and policies.
Time	The student project has a limited development period, so advanced integrations may be deferred.
Data quality	Incorrect, duplicate, or misleading submissions may affect prioritization and require human review.

6. Validate Requirement Completeness and Consistency

6.1 Validation Checklist

Validation Check	Result / Improvement
Stakeholder coverage	Citizens, moderators, administrators, partners, and notification service are represented.
Functional coverage	Core actions from submission to moderation, participation, updates, and reporting are specified.
Non-functional coverage	Performance, security, privacy, usability, reliability, availability, scalability, maintainability, compatibility, and accessibility are addressed.
Ambiguity	Terms such as 'quickly' or 'securely' are translated into measurable targets where practical; targets should be reviewed with stakeholders.
Consistency	Only authorized roles can moderate content or publish official status updates; access rules must match use cases.
Redundancy	Voting, commenting, following, and status updates are defined as separate functions to avoid overlap.
Missing requirements	Before implementation, confirm language support, retention policy, location precision, file-size limits, and escalation process.
Testability	Each FR/NFR has observable behavior or a measurable target that can be checked during testing.

6.2 Suggested Improvements

- Conduct short interviews with citizens, moderators, and partner organizations to validate priorities.
- Create clickable wireframes and test them with a small group before full implementation.
- Define detailed rules for duplicate detection, status transitions, and report escalation.

- Add acceptance criteria and test cases for every high-priority requirement.
- Review privacy, accessibility, and security requirements before deployment.

7. Prioritization and Implementation Plan

A practical first version should prioritize the core workflow and add advanced features after feedback.

Priority	Features	Reason
Must have	Login, submit challenge, browse/search, moderation, status tracking, basic admin controls	Required for the platform's main purpose.
Should have	Voting, comments, following, notifications, partner updates	Improves participation and transparency.
Could have	Duplicate suggestions, map view, trend analytics, multilingual interface	Adds convenience after the core system is stable.
Future	AI-assisted categorization, priority recommendations, integrations with civic systems	Requires evaluation, safeguards, and stakeholder approval.

8. Conclusion

The proposed Digital Platform to Crowdsource Societal Challenges provides a structured way for communities to report issues, discuss solutions, and follow progress. The requirements define the system's main actors, functions, quality expectations, data, interfaces, assumptions, and constraints. Requirement validation and stakeholder feedback will help ensure that the final system is useful, secure, understandable, and realistic to implement.

— End of Assessment —